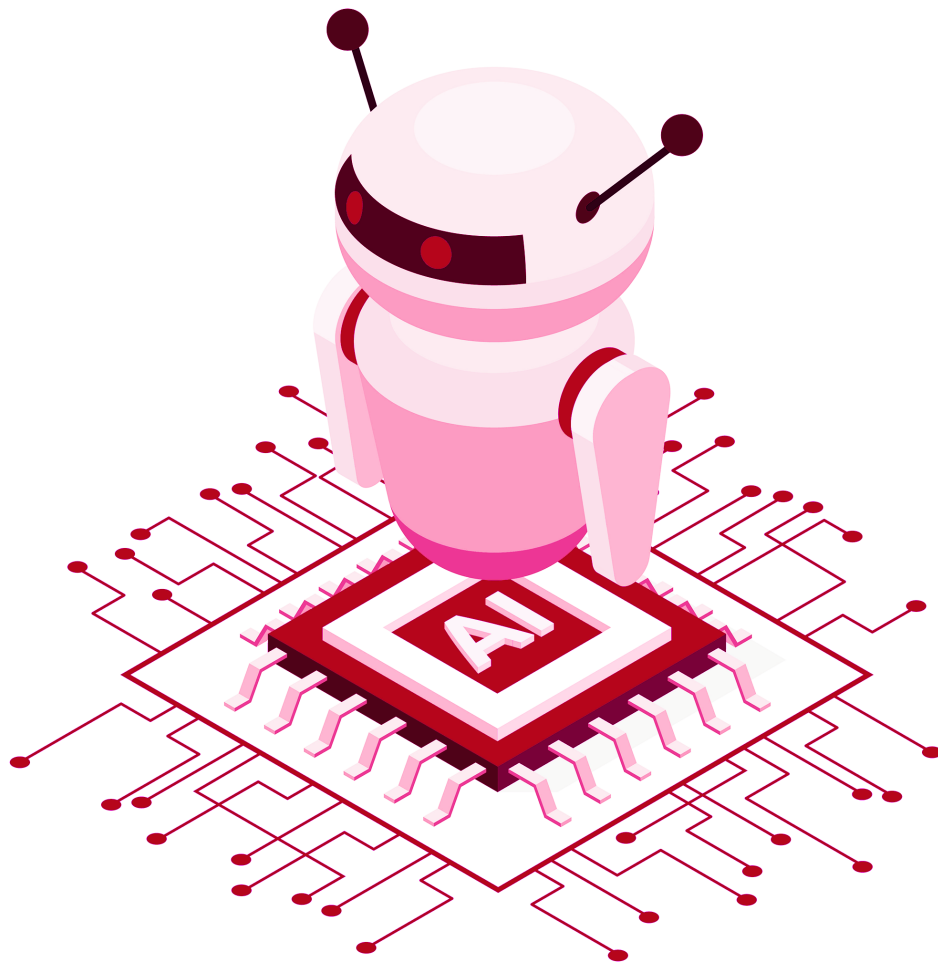




Deep Reinforcement Learning

Professor Mohammad Hossein Rohban

Homework 10:

Exploration in Deep Reinforcement Learning

By:

Parsa Ghezelbash
401110437



Spring 2025

Contents

1 Task 1: Bootstrap DQN Variants

1

2 Task 2: Random Network Distillation (RND)

1

Grading

The grading will be based on the following criteria, with a total of 290 points:

Task	Points
Task 1: Bootstrap DQN Variants	100
Task 2: Random Network Distillation (RND)	100
Clarity and Quality of Code	5
Clarity and Quality of Report	5
Bonus 1	80

1 Task 1: Bootstrap DQN Variants

- The complete guidelines for implementing the Bootstrap DQN algorithm, including the RPF and BIV variants, are provided in the Jupyter notebook. You will find detailed instructions on how to set up the environment, implement the algorithms, and evaluate their performance.
- Make sure to read Guidelines section in the notebook carefully.

2 Task 2: Random Network Distillation (RND)

- You will implement the missing core components of Random Network Distillation (RND) combined with a Proximal Policy Optimization (PPO) agent inside the MiniGrid environment.
- **TODO:** You must complete the following parts:

File	TODO Description
Core/model.py	Implement the architecture of TargetModel and PredictorModel.
Core/model.py	Implement <code>_init_weights()</code> method for proper initialization.
Core/ppo_rnd_agent.py	Implement <code>calculate_int_rewards()</code> to compute intrinsic rewards.
Core/ppo_rnd_agent.py	Implement <code>calculate_rnd_loss()</code> to compute predictor training loss.

Table 1: Summary of required TODO implementations

- Questions:
 1. What is the intuition behind Random Network Distillation (RND)? Why does a prediction error signal encourage better exploration?
 2. Why is it beneficial to use both intrinsic and extrinsic returns in the PPO loss function?
 3. What happens when you increase the `predictor_proportion` (i.e., the proportion of masked features used in the RND loss)? Does it help or hurt learning?
 4. Try training with `int_adv_coeff=0` (removing intrinsic motivation). How does the agent's behavior and reward change?
 5. Inspect the TensorBoard logs. During successful runs, how do intrinsic rewards evolve over time? Are they higher in early training?

Answers to Task 2 Questions

Question 1:

Random Network Distillation (RND) uses curiosity-driven exploration by using two neural networks: a fixed *target* network and a trainable *predictor* network. The predictor tries to mimic the target's output for observed states. The key intuition is that states the agent visits frequently become easier to predict (low error), while novel states yield high prediction errors. This error directly serves as an intrinsic reward signal, encouraging the agent to explore unfamiliar states.

Question 2:

Combining intrinsic and extrinsic returns balances exploration and exploitation. Extrinsic rewards are sparse and task-specific but demands optimal behavior once discovered. Intrinsic rewards provide dense,

exploration-focused signals that help overcome sparse reward problems. Merging them in the PPO loss allows the agent to explore effectively early on using intrinsic motivation and Gradually shift focus to task-specific rewards as they're discovered and Maintains exploration even after finding initial rewards to avoid local optima.

Question 3:

Increasing `predictor_proportion` (masking more features in RND loss) hurts learning. Higher masking reduces the predictor's input information, making state prediction harder and Creates noisy intrinsic rewards that misguide exploration.

Question 4:

Setting `int_adv_coeff=0` (disabling intrinsic motivation) should result in slower exploration, especially in sparse-reward environments and causes Failure to discover complex reward pathways in MiniGrid and converges to suboptimal policies.

As you can see in the below figure, the intrinsic reward decreases if we set `int_adv_coeff=0` but the extrinsic reward maintains its value with a little noise.

Question 5:

In successful runs, intrinsic rewards:

- Stay low as the agent is starting to explore new states and learn them
- Increases when the target network explore and learns a base of states for itself
- Then decay as states become familiar and prediction errors decrease

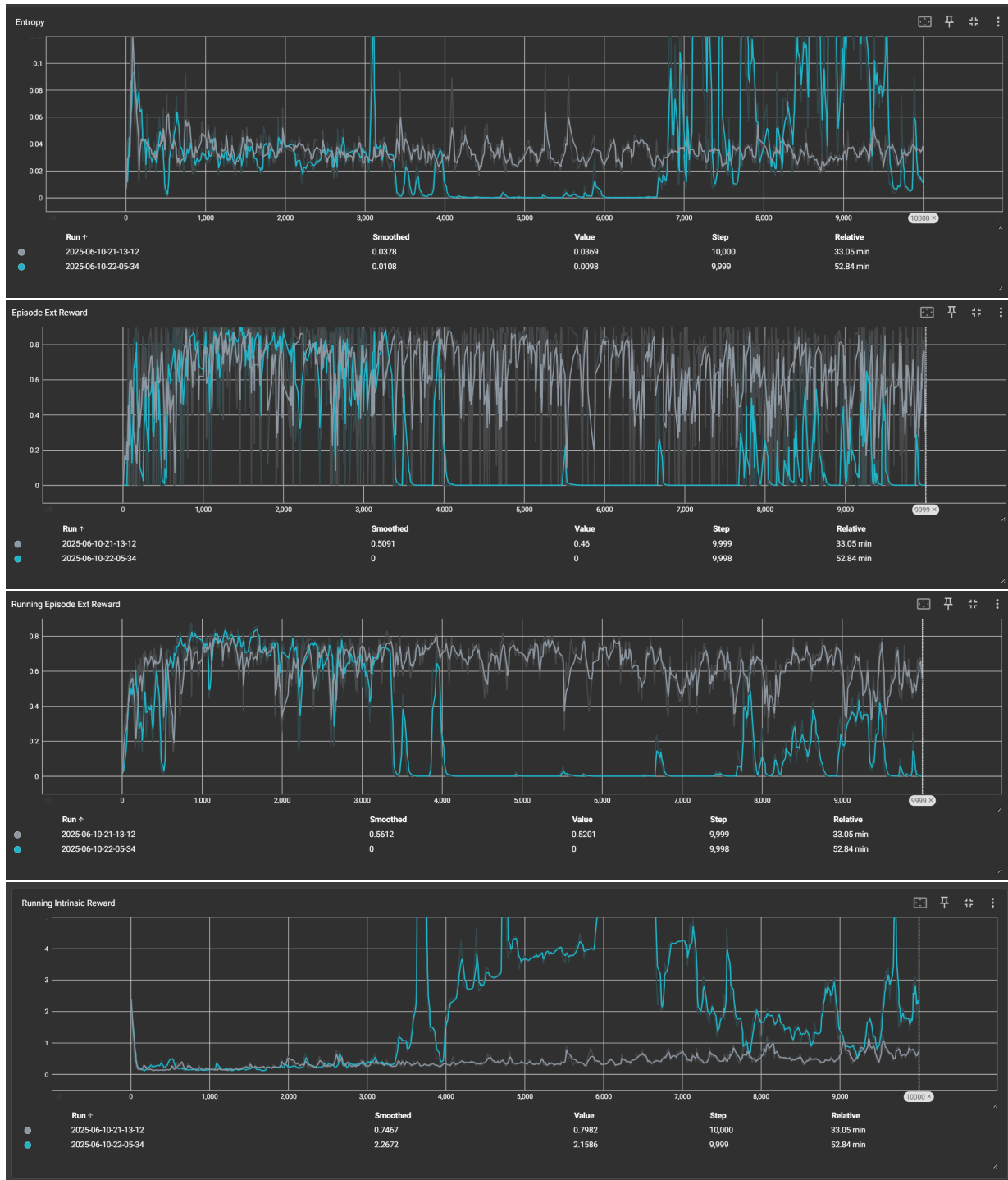


Figure 1: The gray plots are for the `int_adv_coeff=0` and the blue plots are for the `int_adv_coeff=1` run